

420-G66-RO – Services d'échange de données

Examen 1 Modélisation d'un service et bases de données – Automne 2025

Consignes :

- L'examen est réparti en deux sections : une section à compléter sur papier et une section à compléter sur la plateforme Progression.
- La section à résoudre sur papier porte sur la modélisation d'un service d'échange de données. Cette section, se déroule sans accès à l'ordinateur.
- La section à résoudre sur la plateforme Progression porte sur l'implémentation et l'interrogation d'une base de données. Pour cette section, vous avez accès à toutes vos ressources.
- L'utilisation de toute forme d'intelligence artificielle est interdite.
- Tous les outils de communication (courriel, réseau sociaux, chat, y compris Mio) sont également INTERDIT.
- Les téléphones cellulaires doivent être éteints et rangés dans vos sacs.
- Si vous devez aller aux toilettes durant l'examen, vous devez laisser votre téléphone portable sur le bureau de l'enseignant.
- Toutes contraventions aux règles de l'examen, ainsi que toutes tentatives de communication ou de plagiat entraînent automatiquement la note de zéro pour tous les étudiants concernés.

Section 1 – Modélisation d'un service 1 étude de cas à développement Sur cet énoncé, complétez l'ensemble des diagrammes et modèles afin de prendre en compte le nouveau scénario d'utilisation. Temps suggéré : 1h00	8 points
Section 2 – Bases de données - implémentation 5 questions Progression Sur la plateforme Progression, répondez aux questions d'implémentation de la base de données. Temps suggéré : 1h15	8 points
Section 3 – Bases de données - interrogation 2 questions Progression Sur la plateforme Progression, répondez aux questions d'interrogation de la base de données Temps suggéré : 0h45	4 points
20 points	

Section 1 – Modélisation d'un service d'échange de données (8 points)

Scénario

Notre service devient populaire et nous nous rendons compte que notre gestion des abonnements manque de flexibilité. Conserver uniquement la date d'expiration de l'abonnement ne permet pas de faire un suivi de nos usagers membres.

Par exemple, si l'utilisateur Léa s'est abonnée au service pour un mois lors de la création de son compte, le 15 avril 2024, qu'elle n'a pas pris d'abonnement les mois suivants et qu'elle a ensuite pris un nouvel abonnement d'un mois le 1er juillet 2025, nous sommes incapables de le savoir. Lorsque nous consultons ses données, nous connaissons seulement le moment de la création de son compte et la fin de son dernier abonnement.

Afin d'avoir un meilleur portrait de nos usagers, nous devons conserver l'historique de tous les abonnements qu'un usager a achetés depuis la création de son compte. Nous pourrions ainsi produire des statistiques d'abonnement et savoir combien d'abonnés nous avons à une date donnée ou pendant une période de l'année.

Dans notre exemple de Léa, cela signifie qu'une recherche sur son historique d'abonnements nous montrerait deux abonnements : le premier du 15 avril au 14 mai 2024 et le second du 1er au 31 juillet 2025.

Récits utilisateur (2 points)

Du point de vue de l'utilisateur

- En tant qu'utilisateur régulier, j'ai besoin d'un abonnement annuel afin de me rendre au travail en vélo tous les jours.
- En tant qu'utilisateur saisonnier, je préfère la flexibilité d'un abonnement mensuel afin d'utiliser le vélo durant les périodes de l'année qui me plaisent.
- En tant qu'utilisateur occasionnel, j'ai besoin d'un abonnement hebdomadaire afin d'utiliser le service de vélos lorsque je suis de passage à Montréal.

Du point de vue des gestionnaires du service de vélo libre-service

Rédigez un minimum de deux récits différents illustrant les besoins des gestionnaires d'après le scénario.

Historique et suivi d'un usager

- En tant que gestionnaire, j'ai besoin de consulter l'historique complet des abonnements d'un usager depuis la création de son compte afin de connaître les périodes pendant lesquelles il a été abonné.
- En tant que gestionnaire, j'ai besoin de savoir combien de fois un usager s'est réabonné afin de repérer les usagers fidèles.

Statistiques (le but principal du scénario)

- En tant que gestionnaire, j'ai besoin de connaître le nombre d'abonnés à une date donnée afin de mesurer la popularité du service.
- En tant que gestionnaire, j'ai besoin de connaître le nombre d'abonnés pendant une période de l'année afin de comparer l'achalandage entre les saisons.
- En tant que gestionnaire, j'ai besoin de voir la répartition des abonnements par type (annuel, mensuel, hebdomadaire) afin d'ajuster l'offre du service.

Gestion des abonnements

- En tant que gestionnaire, j'ai besoin de consulter la liste des abonnements actifs d'une année donnée afin de produire des rapports pour la fondatrice du projet. (Ça correspond au scénario 2 de la section des requêtes.)
- En tant que gestionnaire, j'ai besoin de savoir quels abonnements arrivent à échéance prochainement afin de relancer les usagers.

Récits d'utilisateur (même s'ils sont déjà donnés, ils servent de modèle)

- En tant qu'utilisateur, j'ai besoin de consulter mes anciens abonnements afin de vérifier ce que j'ai déjà acheté.
- En tant qu'utilisateur, j'ai besoin de renouveler mon abonnement avant son expiration afin de ne pas être sans vélo.

Pour bien les maîtriser

- **Structure fixe** : *En tant que [rôle], j'ai besoin de [action] afin de [raison].* Le « afin de » doit expliquer la valeur de l'action, pas la répéter.

Critères d'acceptation

Un critère décrit comment on vérifie qu'un récit est bien réalisé: **Étant donné** = la situation de départ, **Lorsque** = l'action (la requête API), **Alors** = le résultat (la réponse).

Récit: En tant qu'utilisateur saisonnier, j'ai besoin d'acheter un abonnement mensuel afin de pouvoir utiliser les vélos pendant la saison.

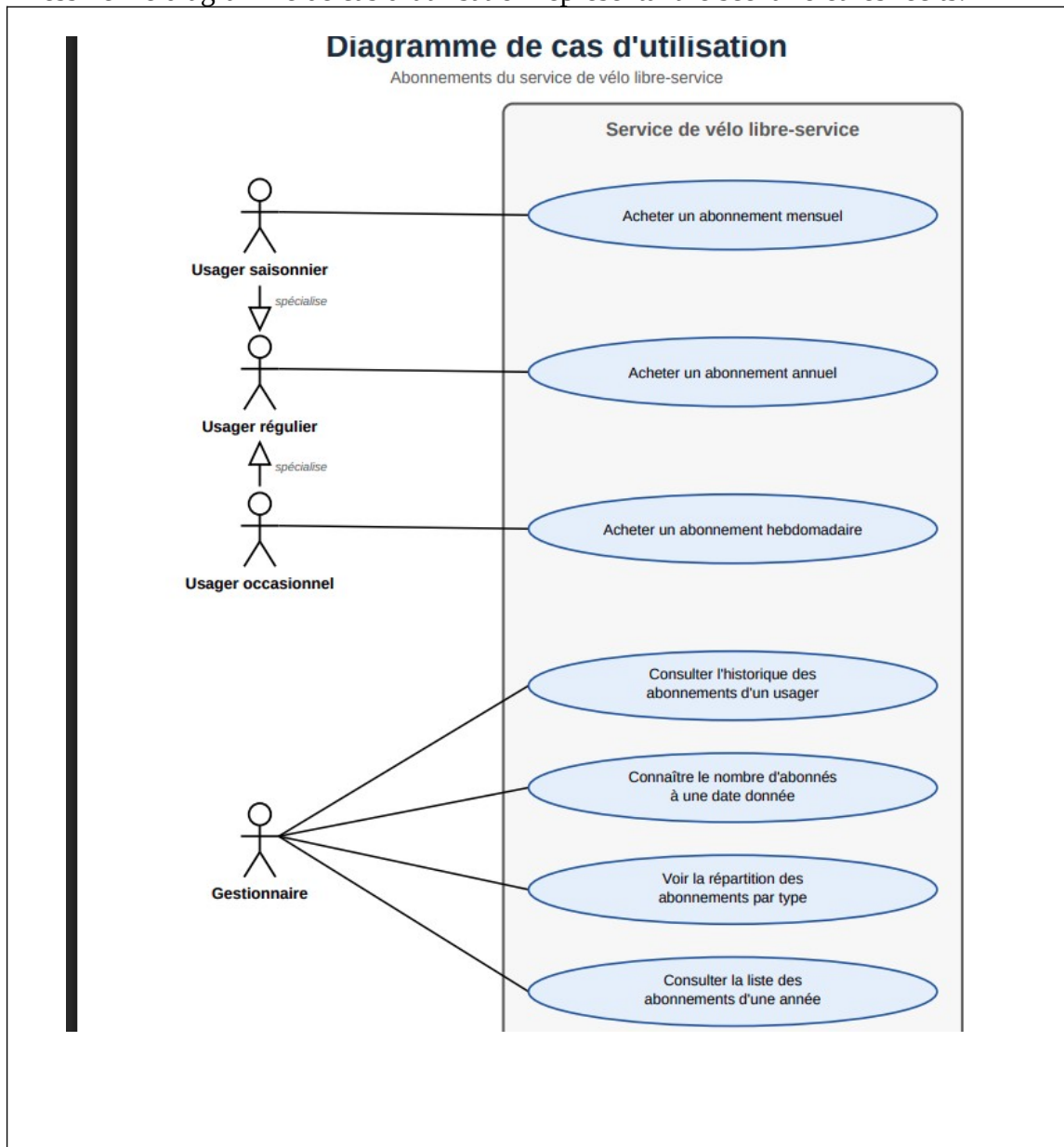
- **Étant donné** qu'un usager est membre du service (numéro de membre 133) et que l'abonnement mensuel existe dans le catalogue
- **Lorsque** il achète un abonnement mensuel avec une date de début (le 15 avril) et que la transaction est acceptée
- **Alors** le système crée la souscription, calcule la date de fin (un mois plus tard, moins un jour) et retourne la confirmation avec le code 201

Récit: En tant que gestionnaire, j'ai besoin de consulter la liste des abonnements d'une année afin de produire des rapports pour la fondatrice du projet.

- **Étant donné** que des abonnements annuels ont été souscrits en 2024
- **Lorsque** le gestionnaire demande la liste des abonnements annuels actifs en 2024
- **Alors** le système retourne la liste des souscriptions correspondantes avec le code 200

Diagramme de cas d'utilisation (2 points)

Dessinez le diagramme de cas d'utilisation représentant le scénario et les récits.



Récits utilisateur correspondants**Du point de vue de l'utilisateur** (donnés dans l'énoncé)

- En tant qu'utilisateur régulier, j'ai besoin d'un abonnement annuel afin de me rendre au travail en vélo tous les jours.
- En tant qu'utilisateur saisonnier, je préfère la flexibilité d'un abonnement mensuel afin d'utiliser le vélo durant les périodes de l'année qui me plaisent.
- En tant qu'utilisateur occasionnel, j'ai besoin d'un abonnement hebdomadaire afin d'utiliser le service de vélos lorsque je suis de passage à Montréal.

Du point de vue des gestionnaires

- En tant que gestionnaire, j'ai besoin de consulter l'historique des abonnements d'un usager afin de connaître les périodes pendant lesquelles il a été abonné depuis la création de son compte.
- En tant que gestionnaire, j'ai besoin de connaître le nombre d'abonnés à une date donnée afin de produire des statistiques d'abonnement.
- En tant que gestionnaire, j'ai besoin de voir la répartition des abonnements par type afin d'ajuster l'offre du service.
- En tant que gestionnaire, j'ai besoin de consulter la liste des abonnements d'une année afin de produire des rapports pour la fondatrice du projet.

Rappels pour redessiner sur papier

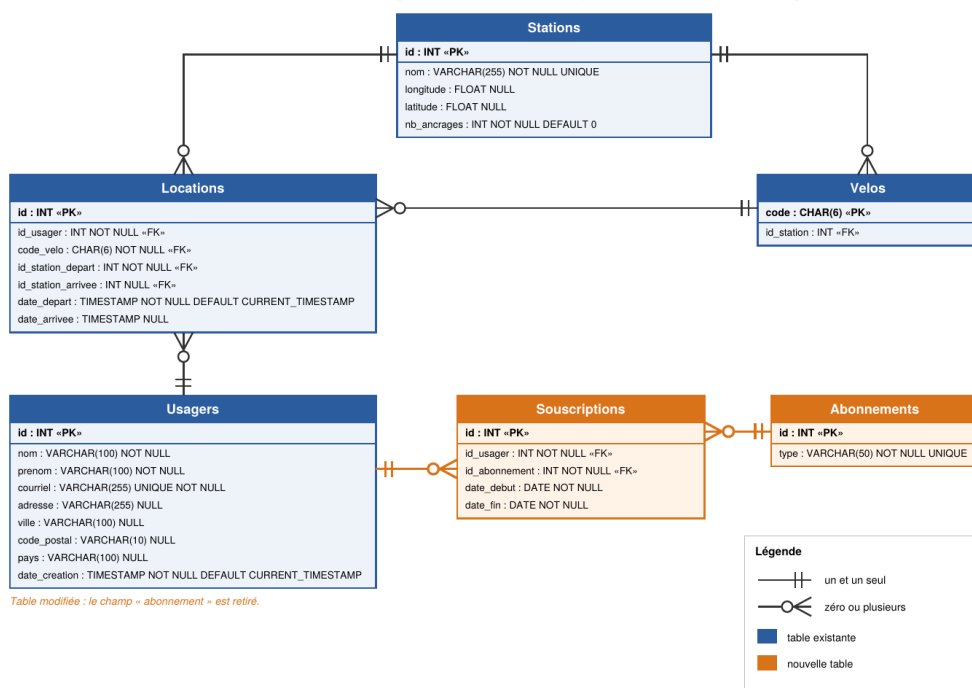
- Le nom du système est écrit en haut du rectangle.
- Chaque récit correspond à une ellipse, et chaque ellipse à un récit.
- Un trait par couple acteur et cas, sans flèche.
- La pointe vide de la généralisation va vers l'acteur général.
- Aucun cas lié aux paiements (hors du projet).

Modèle relationnel des données (2 points)

Modifiez le modèle relationnel de données afin d'y intégrer les informations nécessaires à la gestion des abonnements.

Modèle relationnel : historique des abonnements

Les tables en orange sont nouvelles. Le champ abonnement est retiré de Usagers.



Les deux nouvelles tables

Table	Colonne	Type	Rôle
Abonnements	id	INT «PK»	Identifiant du type
	type	VARCHAR(50) NOT NULL UNIQUE	annuel, mensuel ou hebdomadaire (3 lignes)
Souscriptions	id	INT «PK»	Identifiant de l'achat (clé artificielle)
	id_usager	INT NOT NULL «FK»	Qui a acheté (vers Usagers)
	id_abonnement	INT NOT NULL «FK»	Quel type (vers Abonnements)
	date_debut	DATE NOT NULL	Premier jour de l'abonnement
	date_fin	DATE NOT NULL	Dernier jour de l'abonnement

Exemple : l'historique de Léa

Abonnements		Souscriptions			
id	type	id	id_usager	id_abonnement	date_debut
1	annuel	1	1 (Léa)	2	2024-04-15
2	mensuel	2	1 (Léa)	2	2025-07-01
3	hebdomadaire				2025-07-31

Léa achète deux fois le même type (mensuel) : la paire (id_usager, id_abonnement) se répète. C'est pourquoi la clé primaire est un **id** artificiel et non la paire des deux clés étrangères.

À retenir

- **Abonnements** est le catalogue des types. **Souscriptions** garde chaque achat : c'est l'historique.
- Sur le schéma, on dessine deux relations un-à-plusieurs vers la table du milieu, pas une relation plusieurs-à-plusieurs.
- Un usager a zéro ou plusieurs souscriptions. Une souscription appartient à exactement un usager et à exactement un type.
- La clé étrangère est toujours du côté « plusieurs » : id_usager et id_abonnement sont dans Souscriptions.
- Le champ **abonnement** de Usagers est supprimé : il ne gardait que la fin du dernier abonnement.

Ce que le modèle permet (statistiques)

Nombre d'abonnés à une date donnée :

```
SELECT COUNT(DISTINCT id_usager)
FROM Souscriptions
WHERE date_debut <= '2025-07-15' AND date_fin >= '2025-07-15';
```

420-G66-RO - Modèle relationnel - page 2

Requêtes (2 points)

- Pour les critères d'acceptation suivants, rédigez la requête échangée entre l'application cliente et l'API.
- Concentrez-vous uniquement sur les opérations liées aux abonnements.
- Ignorez les détails des paiements, qui ne sont pas développés dans notre projet.
- Pour les listes d'éléments, écrivez seulement deux éléments, suivis de ... pour signifier que la liste continue.

Scénario 1

Étant donné Marc Lapointe, usager authentifié, membre no 133, lorsque, le 15 avril, il achète un abonnement mensuel actif immédiatement via son application mobile et que la transaction est acceptée,

alors il obtient la confirmation de son abonnement et les détails de celui-ci.

Requête

POST	http://www.serveur.com/api/souscriptions_____
<pre>{ "id_usager": 133, "type": "mensuel", "date_debut": "2025-04-15" }</pre>	

Réponse

	201
<pre>{ "id": 541, "id_usager": 133, "type": "mensuel", "date_debut": "2025-04-15", "date_fin": "2025-05-14" }</pre>	

Scénario 2

Étant donné Andréanne Sauvé, fondatrice du projet authentifié, lorsque qu'elle consulte la liste de des abonnés annuels pour 2024, alors elle obtient la liste de tous les abonnements de type annuel actifs en 2024.

Requête

	http://www.serveur.com/api/souscriptions?type=annuel&annee=2024
--	---

Réponse

	200
<pre>[{ "id": 12, "id_usager": 7, "type": "annuel", "date_debut": "2024-01-10", "date_fin": "2025-01-09" }, { "id": 15, "id_usager": 21, "type": "annuel", "date_debut": "2024-03-01", "date_fin": "2025-02-28" }, ...]</pre>	

PARTIE 2 ET 3

À ne pas oublier dans une colonne (CREATE ou ALTER)

Pour **chaque** colonne, je vérifie ces 6 points:

- **Nom** exact (comme l'énoncé)

- **Type:** INT, VARCHAR(n), CHAR(n), ENUM(. . .), DATE, TIMESTAMP. Avec la longueur (n) si c'est du texte.
- **NOT NULL** si « obligatoire »
- **PRIMARY KEY + AUTO_INCREMENT** si c'est l'id
- **DEFAULT** si l'énoncé donne une valeur par défaut
- **UNIQUE** si « unique »

```
ALTER TABLE Velos ADD COLUMN date_pmes TIMESTAMP NULL;  
UPDATE Velos SET date_pmes = '2024-04-15';  
UPDATE Velos SET date_pmes = '2024-04-15' WHERE code = 'ABC123'; // vec where  
ALTER TABLE Velos MODIFY date_pmes TIMESTAMP NOT NULL;
```

Règles

- Données existantes à conserver: ALTER et UPDATE, jamais DROP.
- « Pour tous les vélos »: UPDATE **sans** WHERE. « Pour certains »: avec WHERE.
- MODIFY redéfinit toute la colonne: j'y réécris le type **et** NOT NULL (sinon je perds la contrainte).
- Même idée pour une clé étrangère obligatoire: colonne facultative, UPDATE, puis NOT NULL.